In [47]:
```
# BAN401
# Week 10
# Case Study Project
# Author: Nathanael Ochoa
# 03/01/26
```

Case Study Steps

I hope this document is easy to follow. The steps are in bold, and I have included observations and brief explanations throughout. I understand that not everyone may be tech-savvy, so I created the steps in Jupyter Notebook alongside the code to make the process clearer and easier to follow.

1. Data Cleaning

a. Import necessary packages.

In [48]:
```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

b. Upload the dataset.

In [49]:
```
df = pd.read_csv("Churn_Modeling.csv")

df_copy = df.copy()  # Will be used for EDA
```

c. Check for any empty values within the dataset.

In [50]:
```
df.isnull().sum()
```

Out[50]:
```
RowNumber          0
CustomerId         0
Surname            0
CreditScore        0
Geography          0
Gender             0
Age                0
Tenure             0
Balance            0
NumOfProducts      0
HasCrCard          0
IsActiveMember     0
EstimatedSalary    0
Exited             0
dtype: int64
```

d. Check the data types of the columns within the dataset.

In [51]:
```
df.dtypes
```

Out[51]:
```
RowNumber            int64
CustomerId           int64
Surname             object
CreditScore          int64
Geography           object
Gender              object
Age                  int64
Tenure               int64
Balance            float64
NumOfProducts        int64
HasCrCard            int64
IsActiveMember       int64
EstimatedSalary    float64
Exited               int64
dtype: object
```

Everything looks good, so we will continue.

The following columns will be removed: *RowNumber*, *CustomerID*, and *Surname*. These columns do not add value to our analysis, and it is best practice to exclude personal identifiers.

e. Remove irrelevant columns.

In [52]:
```
df = df.drop(columns = ["RowNumber", "CustomerId", "Surname"])
```

There are a couple of categorical variables to handle before moving on. This means they are not numerical, so we will encode them, which is a fancy way of saying they will be converted into a numerical format for modeling. It is similar to how 1 can mean yes and 0 can mean no in binary terms. For the *Gender* variable, 1 will replace "Male" and 0 will replace "Female." The get_dummies function will be used for the *Geography* variable.

f. Encode Categorical values.

In [53]:
```
# Binary encoding for Gender
df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})
```

```
# One-hot encoding for Geography
df = pd.get_dummies(df, columns = ["Geography"], drop_first = True)
```

g. Check for duplicates.

In [54]:
```
df.duplicated().sum()
```

Out[54]:
```
0
```

h. Define features and target variable.

In [55]:
```
X = df.drop("Exited", axis = 1)  # Independent variables
y = df["Exited"]  # Dependent variable
```

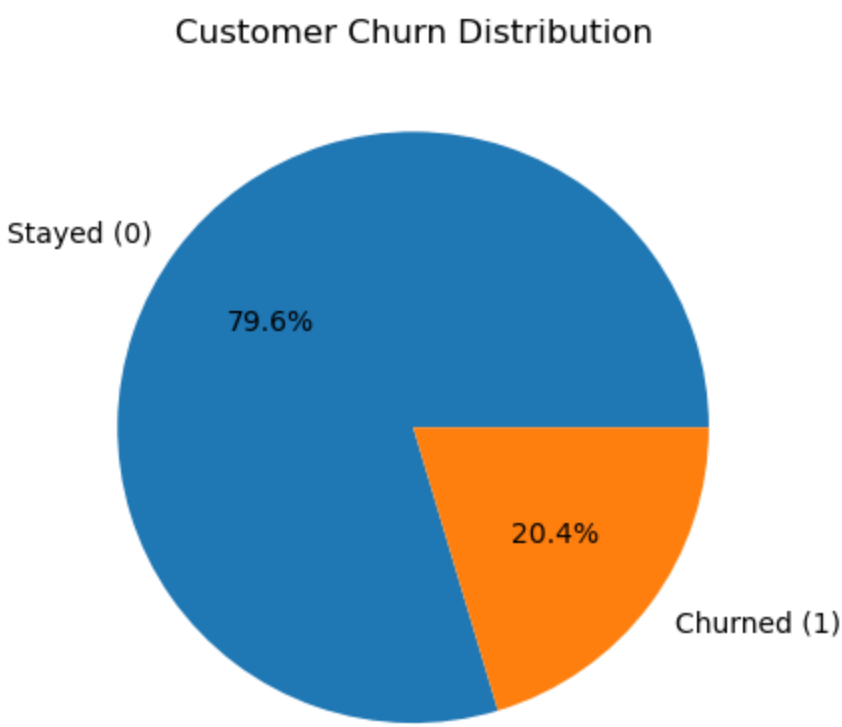
2. Exploratory Data Analysis (EDA)

a. First, examine whether churn is balanced in the dataset. An 80/20 split is generally ideal, with approximately 80% of customers staying and 20% exiting.

In [56]:
```
counts = y.value_counts()

plt.figure()
plt.pie(counts, labels = ["Stayed (0)", "Churned (1)"], autopct = "%1.1f%%")
plt.title("Customer Churn Distribution")

plt.show()
```

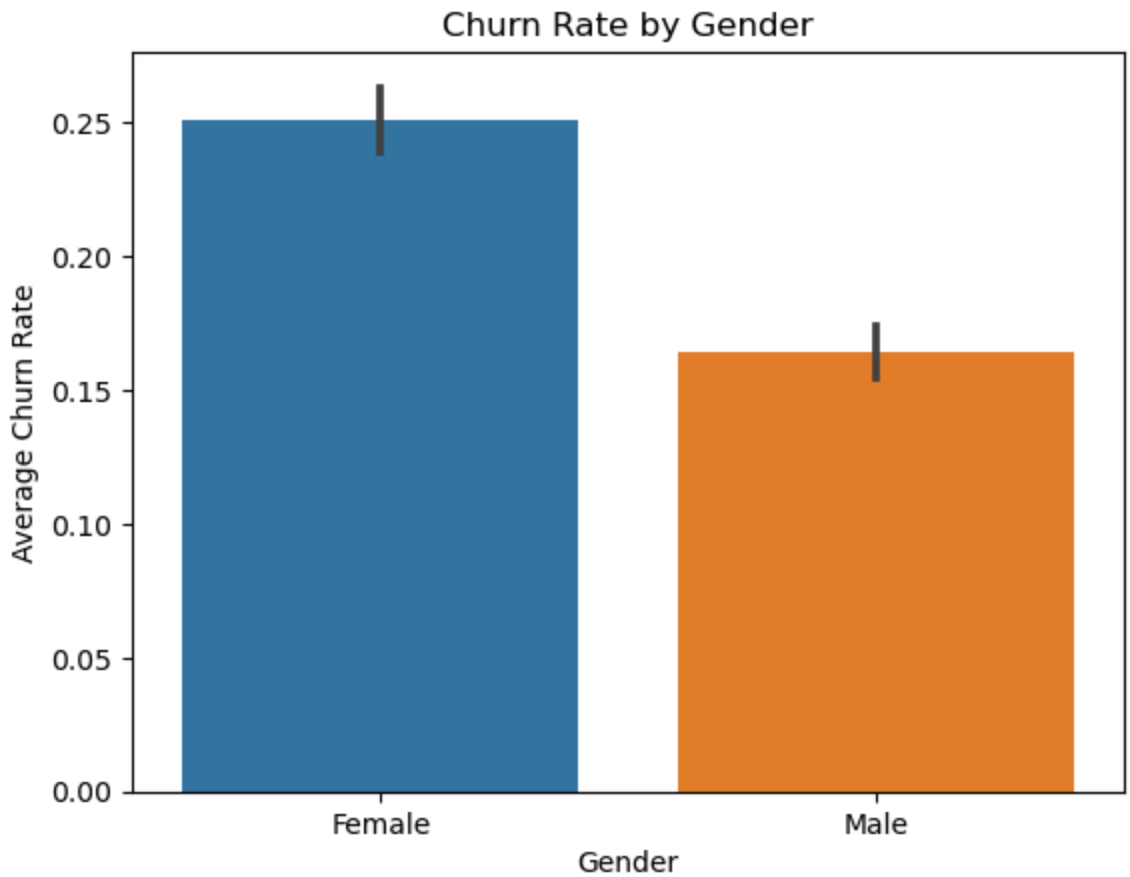


b. Plot average churn rate by *Gender*.

In [57]:
```
plt.figure()
sns.barplot(x = "Gender", y = "Exited", data = df)

plt.xticks([0,1], ["Female", "Male"])
plt.title("Churn Rate by Gender")
plt.xlabel("Gender")
plt.ylabel("Average Churn Rate")

plt.show()
```



It seems that females have a higher churn percentage when compared to males.

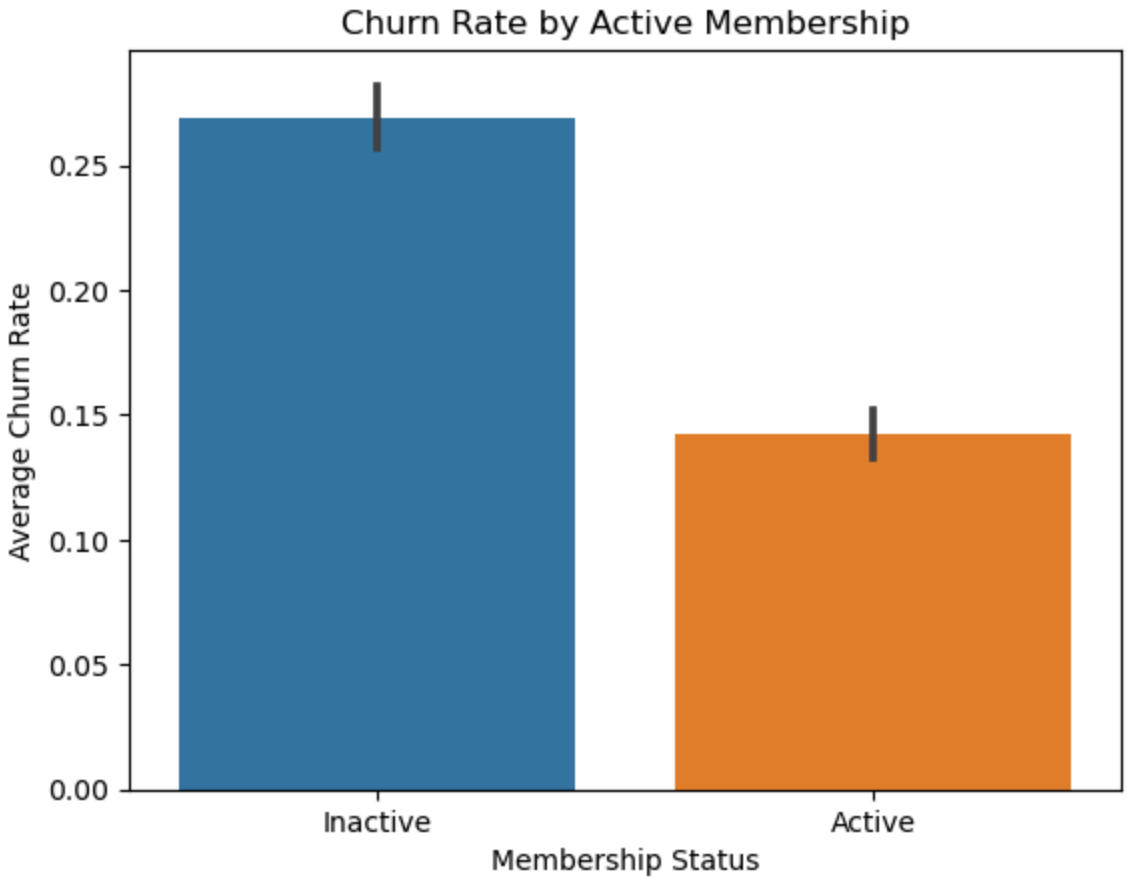
c. Plot average churn rate by *IsActiveMember*.

Now, it is not specified what it means for a client to be an active member. It likely represents whether the client's account usage or transactional activity qualifies them as active.

In [58]:
```
plt.figure()
sns.barplot(x = "IsActiveMember", y = "Exited", data = df)

plt.xticks([0,1], ["Inactive", "Active"])
plt.title("Churn Rate by Active Membership")
plt.xlabel("Membership Status")
plt.ylabel("Average Churn Rate")

plt.show()
```



Clearly, inactive members have much higher churn rates. It is interesting to note that there is still some churn among clients who were considered active members.

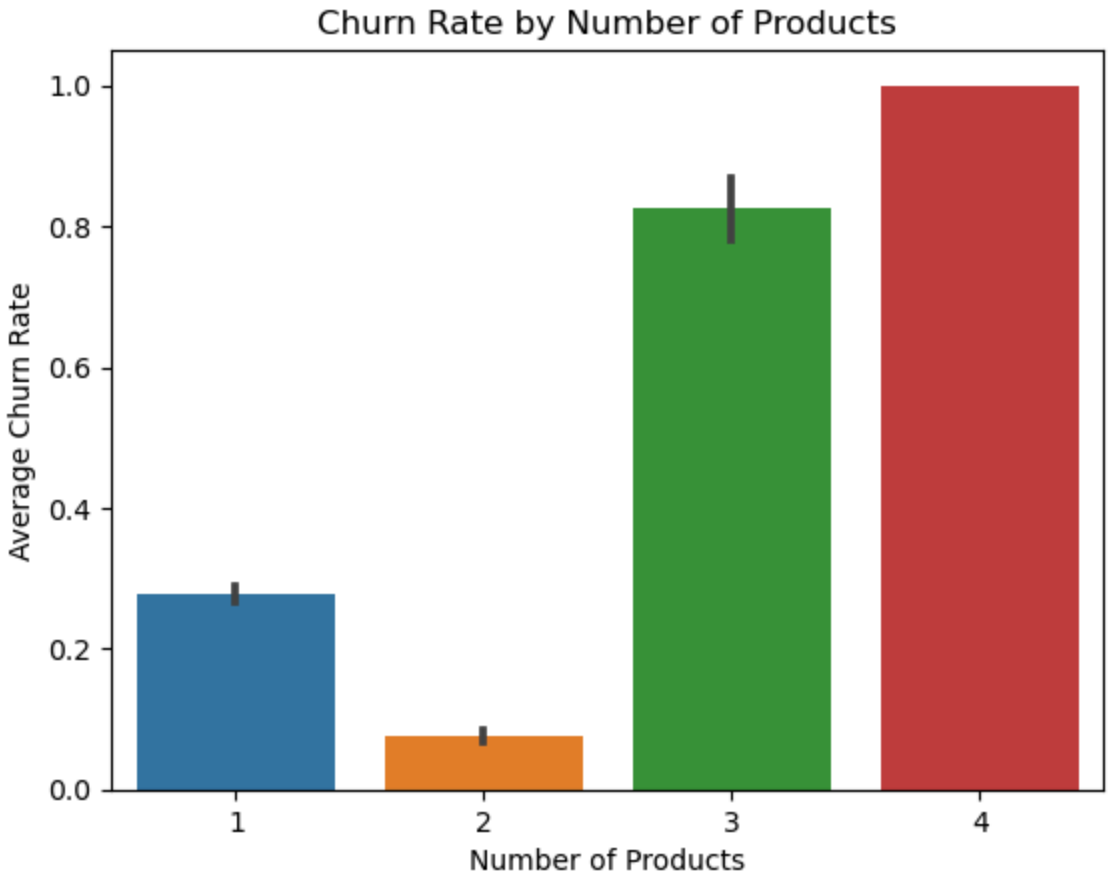
d. Plot average churn rate by *NumOfProducts*.

Perhaps customers with fewer products churn more.

In [59]:
```
plt.figure()
sns.barplot(x = "NumOfProducts", y = "Exited", data = df)

plt.title("Churn Rate by Number of Products")
plt.xlabel("Number of Products")
plt.ylabel("Average Churn Rate")

plt.show()
```



While it may seem that almost all clients with 4 products left the bank, this chart can actually be misleading. It is possible that there are only a small number of clients with 4 products at the bank. For example:

In [60]:
```
df["NumOfProducts"].value_counts()
```

Out[60]:
```
NumOfProducts
1    5084
2    4590
3     266
4      60
Name: count, dtype: int64
```

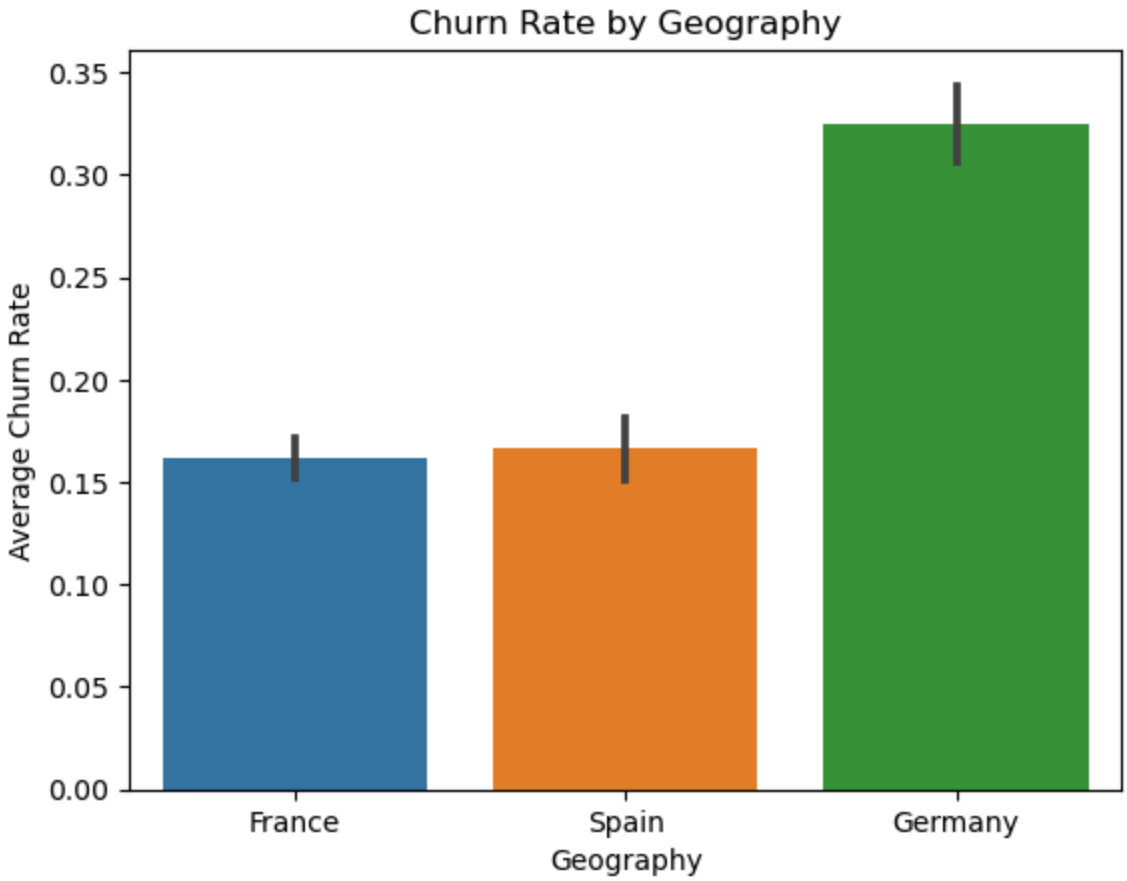
The core banking clientele falls within the 1 to 2 product range. It is possible that the few clients with 3 or more products did not like having to manage so many products and decided to leave the bank altogether.

e. Plot average churn rate by *Geography*

In [61]:
```
plt.figure()
sns.barplot(x = "Geography", y = "Exited", data = df_copy)  # Copy used here

plt.title("Churn Rate by Geography")
plt.xlabel("Geography")
plt.ylabel("Average Churn Rate")

plt.show()
```



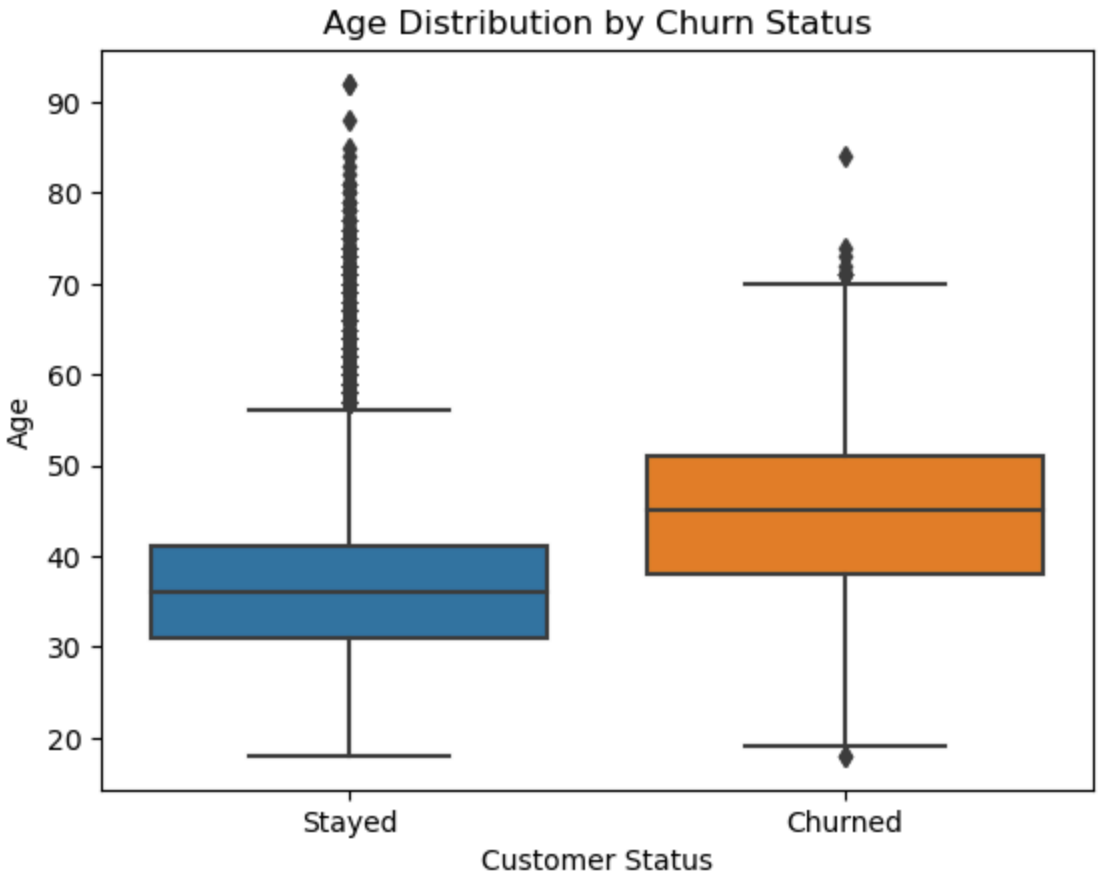
It is interesting to see that clients in Germany have a much higher average churn rate.

f. Plot *Age* vs Churn

In [62]:
```
plt.figure()
sns.boxplot(x = "Exited", y = "Age", data = df)

plt.xticks([0,1], ["Stayed", "Churned"])
plt.title("Age Distribution by Churn Status")
plt.xlabel("Customer Status")
plt.ylabel("Age")

plt.show()
```



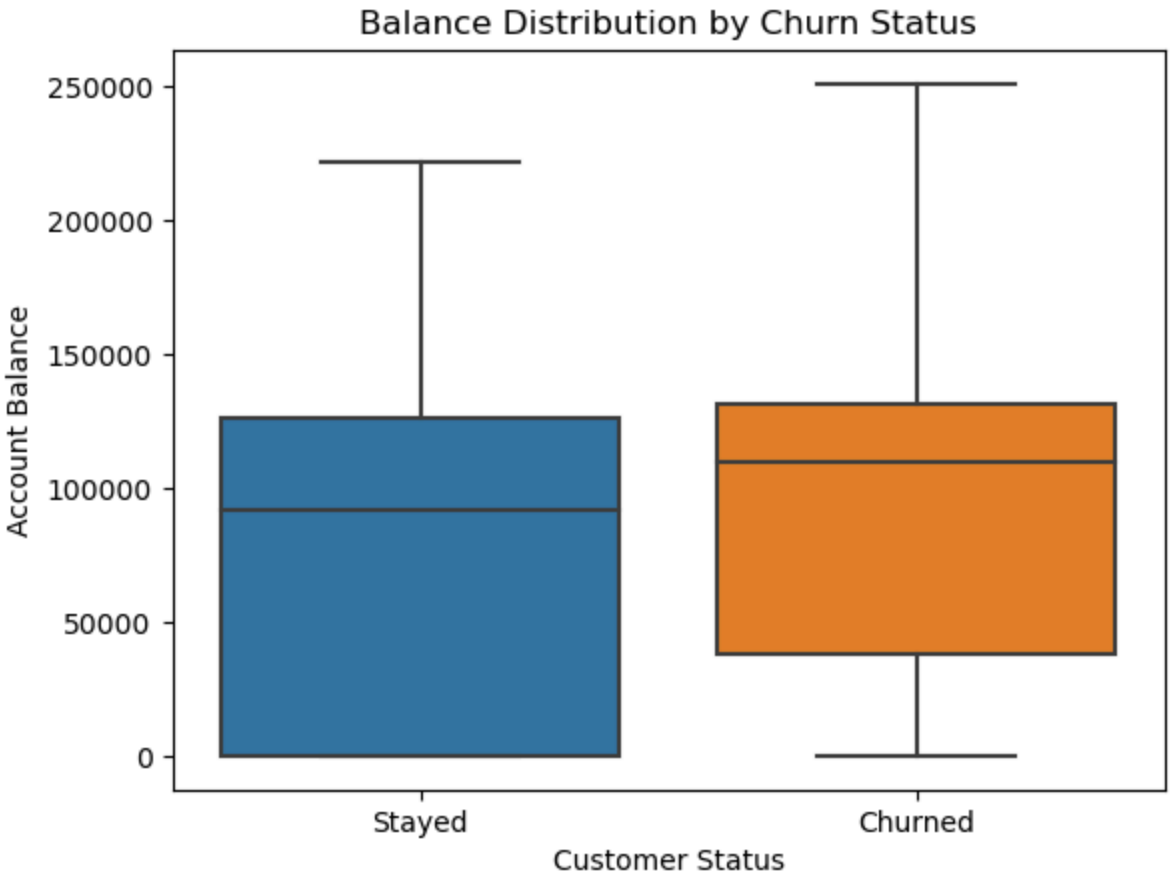
It seems that clients who churned tend to be older than those who remained with the bank.

g. Plot *Balance* vs Churn

In [63]:
```
plt.figure()
sns.boxplot(x = "Exited", y = "Balance", data = df)

plt.xticks([0,1], ["Stayed", "Churned"])
plt.title("Balance Distribution by Churn Status")
plt.xlabel("Customer Status")
plt.ylabel("Account Balance")

plt.show()
```



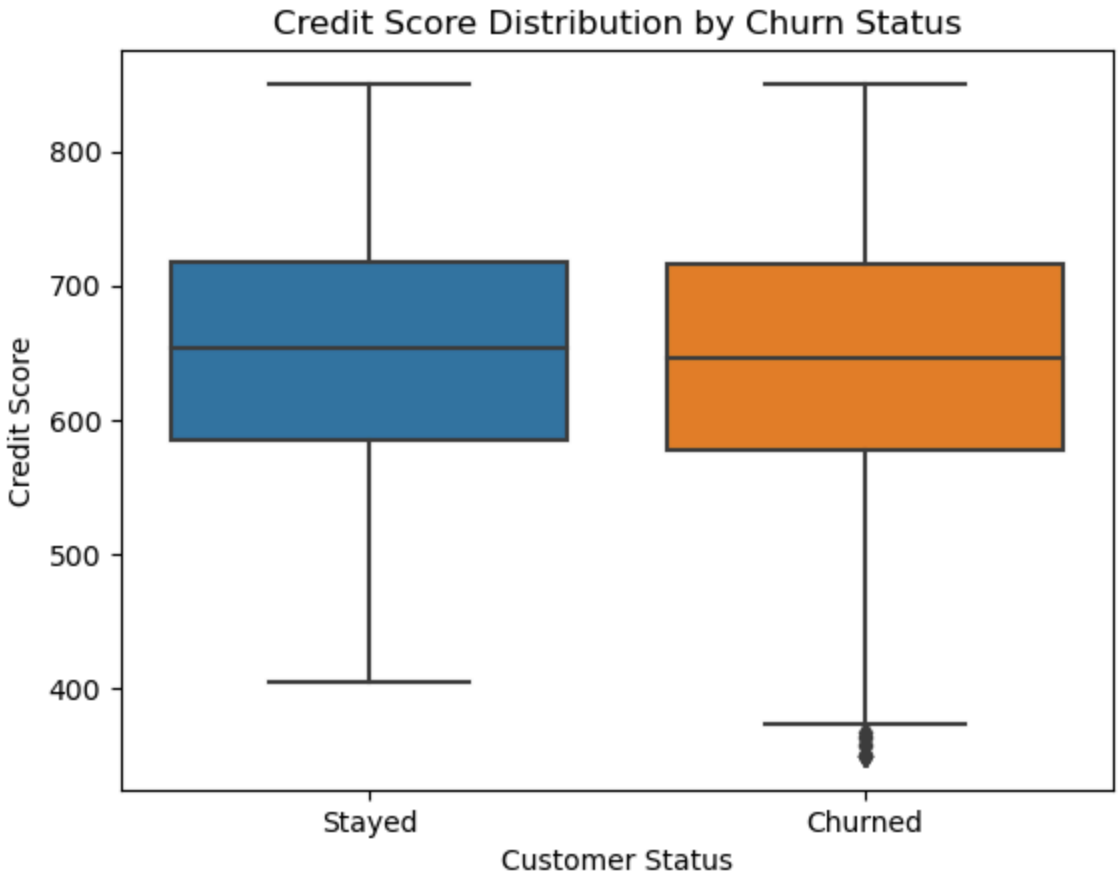
Customers who churned tend to have higher account balances compared to those who remained. It is possible that customers with higher balances are more likely to move to competitors that offer better rates.

h. Plot *CreditScore* vs Churn

In [64]:
```
plt.figure()
sns.boxplot(x = "Exited", y = "CreditScore", data = df)

plt.xticks([0,1], ["Stayed", "Churned"])
plt.title("Credit Score Distribution by Churn Status")
plt.xlabel("Customer Status")
plt.ylabel("Credit Score")

plt.show()
```



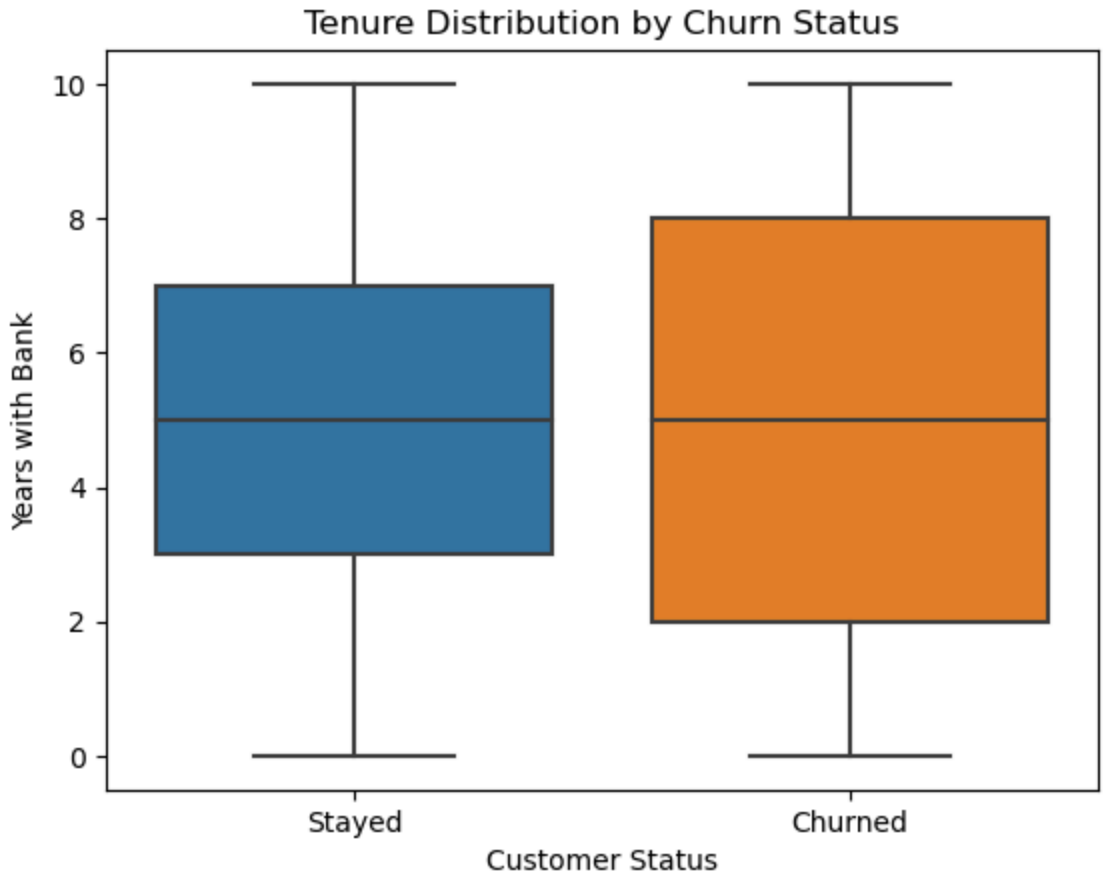
It seems that there is hardly any difference in credit scores between customers who churned and those who remained. This variable may not be very important in the long run.

i. Plot *Tenure* vs Churn

In [65]:
```
plt.figure()
sns.boxplot(x = "Exited", y = "Tenure", data = df)

plt.xticks([0,1], ["Stayed", "Churned"])
plt.title("Tenure Distribution by Churn Status")
plt.xlabel("Customer Status")
plt.ylabel("Years with Bank")

plt.show()
```



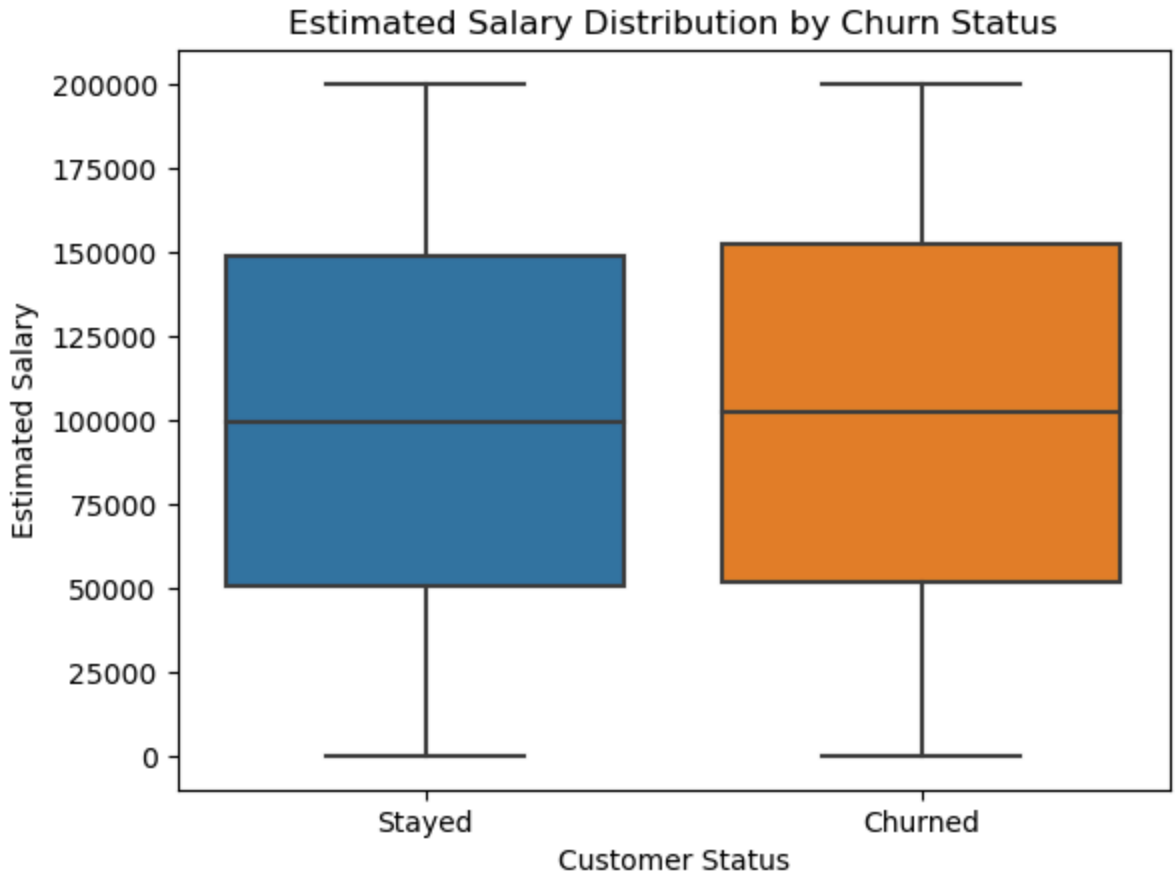
The boxplot shows some overlap; however, there is no strong visual separation between the two groups.

j. Plot *EstimatedSalary* vs Churn

In [66]:
```
plt.figure()
sns.boxplot(x = "Exited", y = "EstimatedSalary", data = df)

plt.xticks([0,1], ["Stayed", "Churned"])
plt.title("Estimated Salary Distribution by Churn Status")
plt.xlabel("Customer Status")
plt.ylabel("Estimated Salary")

plt.show()
```



This boxplot is similar to the Credit Score boxplot. There is virtually no difference, and I do not believe this variable will prove to be very impactful.

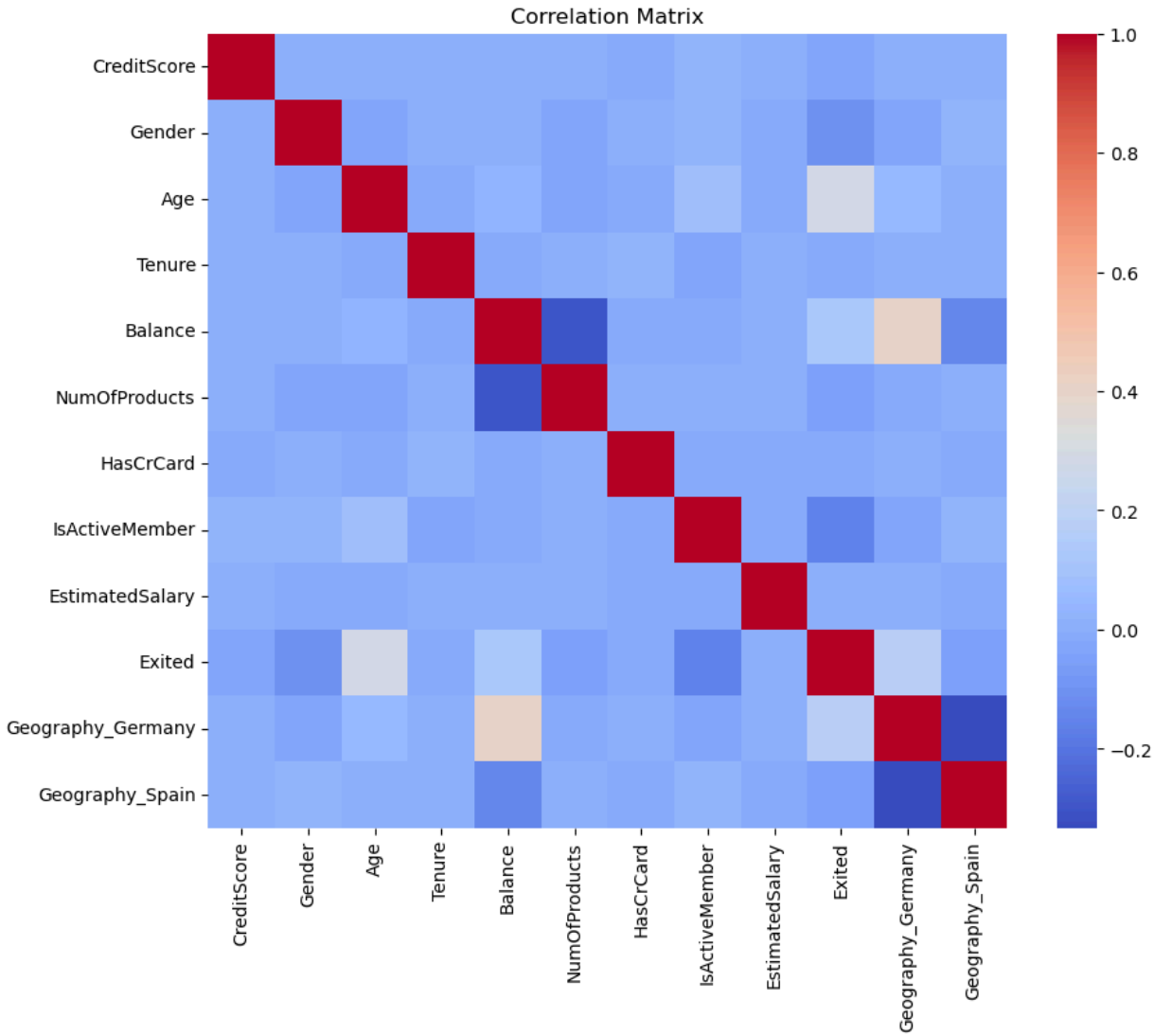
k. Create a correlation matrix

Focus on the *Exited* variable in the following matrix.

In [67]:
```
plt.figure(figsize = (10,8))
sns.heatmap(df.corr(), cmap = "coolwarm")

plt.title("Correlation Matrix")

plt.show()
```



Based on the matrix alone, there do not appear to be any strong correlations between the independent variables and the dependent variable. There is some color variation between *Age* and *Exited*, so let's see if it amounts to anything.

3. Modeling

We will be using logistic regression because it is well suited for binary classification models, and its coefficients are very useful for interpretation. First, the dataset must be split into training and testing sets to evaluate how well the model generalizes to unseen data. The model is trained on the training set and then evaluated on the testing set to measure accuracy and overall performance.

a. Create the train-test split (Using the X and y datasets from earlier).

In [68]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state = 26,
                                                    stratify = y)
```

Feature scaling is used to prevent variables with larger values from dominating the model, such as the difference between *Balance* and *Age*.

b. Apply feature scaling.

In [69]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

c. Train the model.

In [70]:
```
log_model = LogisticRegression(max_iter = 1000)

log_model.fit(X_train_scaled, y_train)
```

Out[70]:
```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

d. Make predictions

In [71]:
```
y_pred = log_model.predict(X_test_scaled)
```

e. Evaluate the model

In [72]:
```
print(confusion_matrix(y_test, y_pred))
```

```
[[1520   73]
 [ 316   91]]
```

The model correctly predicted that 1,520 customers would stay and that 91 would churn. There were 73 false positives, meaning those customers were predicted to churn but actually stayed. The model failed to identify 316 customers who actually churned.

In [73]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.83      0.95      0.89      1593
           1       0.55      0.22      0.32       407

    accuracy                           0.81      2000
   macro avg       0.69      0.59      0.60      2000
weighted avg       0.77      0.81      0.77      2000
```

The model is very good at predicting customers who stay; however, it only identifies about 22% of actual churners. When it predicts churn, it is correct 55% of the time.

In [74]:
```
roc_auc = roc_auc_score(y_test, log_model.predict_proba(X_test_scaled)[:,1])
print("ROC-AUC Score:", roc_auc)
```

```
ROC-AUC Score: 0.7702155159782278
```

The logistic regression model achieved a ROC-AUC score of 0.77, which indicates good discriminative performance.

f. Extract coefficients

In [75]:
```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": log_model.coef_[0]
}).sort_values(by = "Coefficient", ascending = False)
```

In [76]:
```
coefficients
```

Out[76]:

	Feature	Coefficient
2	Age	0.764836
9	Geography_Germany	0.336556
4	Balance	0.159047
8	EstimatedSalary	0.029969
6	HasCrCard	0.003453
10	Geography_Spain	-0.013906
3	Tenure	-0.040854
5	NumOfProducts	-0.043186
0	CreditScore	-0.051129
1	Gender	-0.268400
7	IsActiveMember	-0.527653

The model indicates that *Age* is the strongest predictor of churn. *Geography* also appears to play a role; however, customers in Germany exhibited a higher likelihood of churn and should be investigated further. *Balance* was the third strongest predictor, although its impact is much smaller than that of *Age*.

Now let's add a Random Forest Model to compare.

4. Comparison

a. Random Forest does not require scalling so we will use the original X_train and X_test.

In [77]:
```
rf_model = RandomForestClassifier(n_estimators = 100, random_state = 26)

rf_model.fit(X_train, y_train)
```

Out[77]:
```
▾ RandomForestClassifier
RandomForestClassifier(random_state=26)
```

b. Generate predictions

In [78]:
```
y_pred_rf = rf_model.predict(X_test)
```

c. Evaluate the model

In [79]:
```
print(confusion_matrix(y_test, y_pred_rf))
```

```
[[1534   59]
 [ 221  186]]
```

The model correctly predicted that 1,534 customers would stay and that 186 would churn. There were 59 false positives, and the model failed to identify 221 customers who actually churned. Based on this alone, the model appears to have performed better than the logistic regression model.

In [80]:
```
print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support

           0       0.87      0.96      0.92      1593
           1       0.76      0.46      0.57       407

    accuracy                           0.86      2000
   macro avg       0.82      0.71      0.74      2000
weighted avg       0.85      0.86      0.85      2000
```

The recall in this model more than doubled compared to the previous model. Precision also improved significantly, as did the F1-score.

In [81]:
```
roc_auc_rf = roc_auc_score(y_test, rf_model.predict_proba(X_test)[:,1])
print("Random Forest ROC-AUC:", roc_auc_rf)
```

```
Random Forest ROC-AUC: 0.8554987961767623
```

The random forest model outperformed the logistic regression model. It achieved a ROC-AUC score of 0.86, indicating superior discriminative performance.

Now, let's display feature importance.

In [82]:
```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
}).sort_values(by = "Importance", ascending = False)
```

In [83]:
```
feature_importance
```

Out[83]:

	Feature	Importance
2	Age	0.239718
0	CreditScore	0.144607
8	EstimatedSalary	0.144415
4	Balance	0.138502
5	NumOfProducts	0.129474
3	Tenure	0.082821
7	IsActiveMember	0.042598
9	Geography_Germany	0.027168
1	Gender	0.018926
6	HasCrCard	0.018116
10	Geography_Spain	0.013655

Age reamined the most important predictor across both modeling approaches.

In [84]:

```
# Clean data to upload for submission:
df.to_csv("cleaned_churn_data.csv", index = False)
```